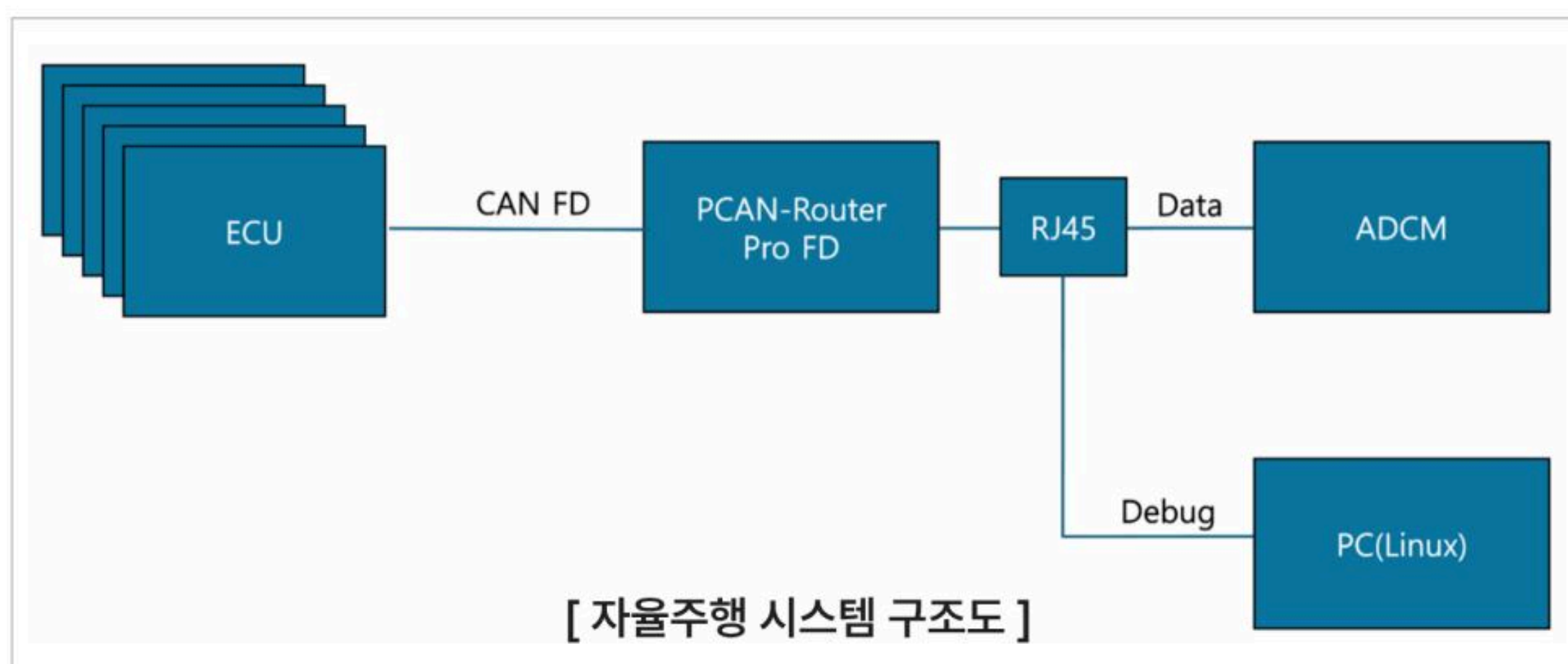


자율주행 차량 CAN Router 기반 Control Unit 개발

Github [GitHub - jaejunmin408/CAN-router-Control-Unit-Bootloader-](https://github.com/jaejunmin408/CAN-router-Control-Unit-Bootloader-)



주요 기능

1. Linux PC를 이용한 Debug 환경(Ethernet)
 - 1-1. Linux PC를 이용해 CAN Router 제어
 - 1-2. Linux PC를 통해 Debug 출력
2. 자율주행시스템(ADCM)과 차량 내 ECU간 통신
 - 2-1. ECU와 CAN Router 간 CAN FD 통신
 - 2-2. ADCM과 CAN Router 간 차량 내 Ethernet 통신

문제 상황

1. 제한적인 차량 환경
 - 1-1. 차량 내부에 CAN Router 설치 → 물리적인 전원 ON/OFF 불가
 - 1-2. 차량 내 Ethernet 환경 → Ethernet 환경에서 CAN Router FW 업데이트
2. CAN to Ethernet 통신 구조 설계

해결 방안

1. 자체 복구 중심의 SW 설계
 - 1-1. CRC 기반 FW 무결성 검증, WDG 기반 비정상 동작 감지, Backup 구조 설계
→ 별도의 전원 제어 없이 시스템이 중단되지 않고 지속적으로 동작하는 환경 구축
2. UDP를 활용한 CAN 메시지 변환 구조 설계
→ 실시간성 문제 해결을 위해 구조 개선 진행중(RTOS 구조)

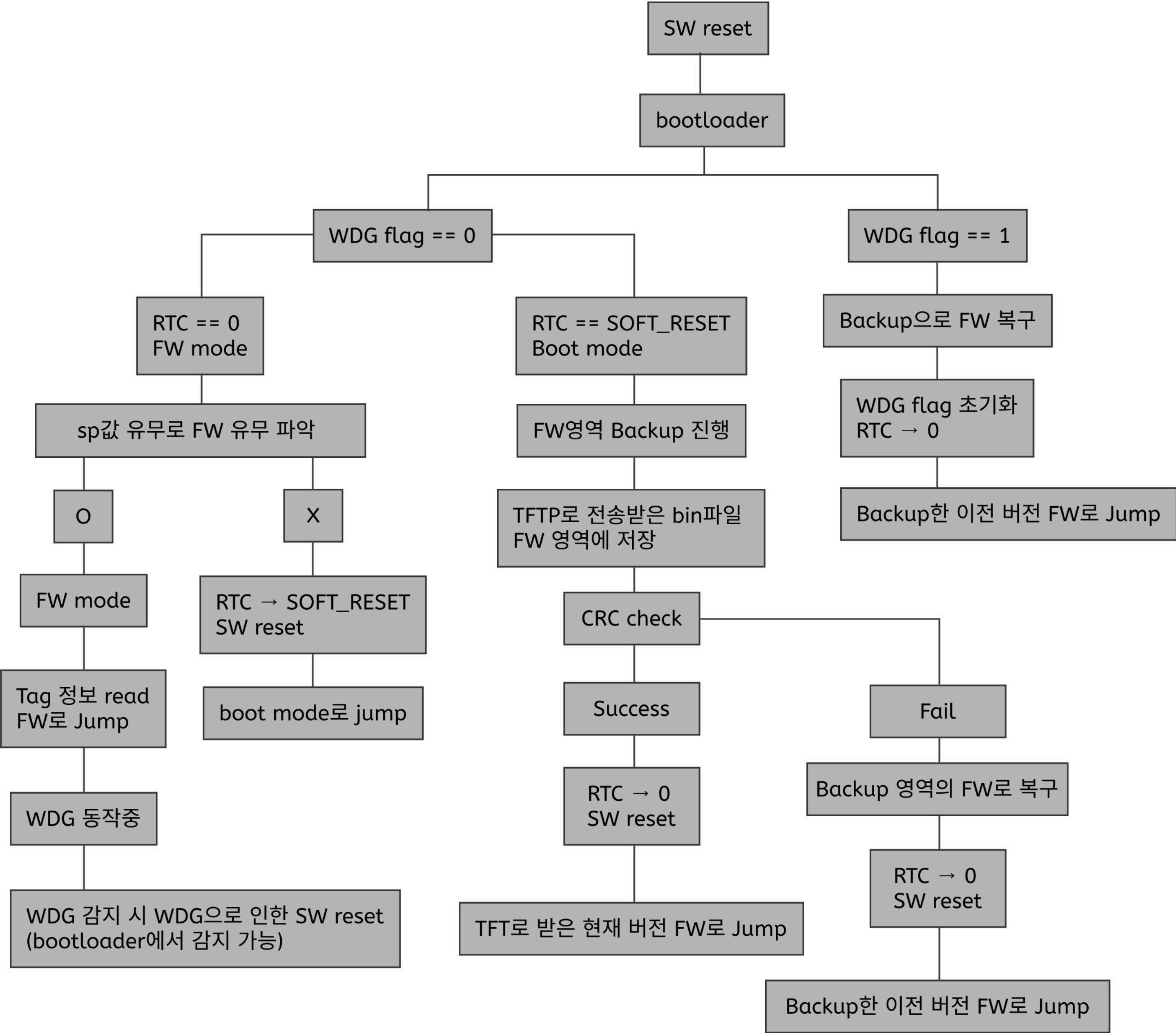
자율주행 차량 CAN Router 기반 Control Unit 개발

세부 개발 사항 1 - 전체 SW 구조 설계

전원 제어가 불가능한 환경에서 WDG, RTC, CRC, Backup 등을 활용해 SW만으로도 복구 가능한 부트 흐름을 설계

< Software-only Recovery Boot Flow (WDG / CRC / Backup 기반) >

* RTC memory == SOFT_RESET : Boot mode , RTC memory == 0 : Boot mode



자율주행 차량 CAN Router 기반 Control Unit 개발

세부 개발 사항 2 - Flash 구조 설계

CAN Router 장비의 Flash 구조를 직접 설계
→ flash 구조 분석 및 Id 파일 수정

0x08000000	32K	peak사 기본 설정
0x08008000	32K	
0x08010000	32K	
0x08018000	32K	custom bootloader
0x08020000	128K	
0x08040000	256K	FW
0x08080000	256K	
0x080C0000	256K	
0x08100000	256K	Backup
0x08140000	256K	
0x08180000	256K	
0x081C0000	256K	

[Flash 메모리 영역]

```
12  #define USER_FLASH_FIRST_PAGE_ADDRESS 0x08040000 /*
13  #define USER_FLASH_LAST_PAGE_ADDRESS  0x080CFFFF
14  #define BACKUP_FLASH_FIRST_PAGE_ADDRESS 0x08100000
15  #define BACKUP_FLASH_LAST_PAGE_ADDRESS 0x081BFFFF
16  #define USER_FLASH_END_ADDRESS        0x081BFFFF
```

[Flash 영역 설정 범위]

Backup 공간을 분리해 업로드한 FW에서 문제가 발생했을때 직전에 동작한
안정적인 Backup 코드로 복구할 수 있도록 구상함

자율주행 차량 CAN Router 기반 Control Unit 개발

세부 개발 사항 3 - Backup 영역의 안전성

- Debug PC를 통해 Bootmode에 돌입해야만 현재 FW영역의 Backup이 진행되도록 설계
→ 제대로 동작하는 FW만 Backup이 진행됨
- 안전성 문제(WDG, CRC)가 있는 FW는 SW적으로 Backup 영역의 FW로 복구됨

```
[BOOT] START BOOT...
[BOOT] Backup : src.bin
[BOOT] New : src3.bin
[BOOT] Update FW...
[BOOT] CRC same, Jump to new FW
[BOOT] Reset the board
```

[BOOT 진행 과정(안정적인 FW를 수신받았을때)]

```
[FW] JUMP FW(RESET)
---FW INFO---
---Filename: src.bin
---Size: 103272 bytes
---Version: 65537
[FW] START FW
```

[FW 진행 과정]

```
min@DESKTOP-50MIIII:~/test/udp_test$ ./client 192.168.20.72 8080

=== Command Menu ===
1. start FW
2. start BOOT
q. Quit
Select :
```

[Debug PC에서의 제어 명령]

세부 개발 사항 4 - RTC flag 설정

- RTC 메모리에 flag를 작성하고(좌측이미지), RTC 메모리영역을 읽어 BOOT, FW mode를 판별함(우측 이미지)

```
#define BOOT_FLAG_SOFT_RESET 0xA5A5
#define BOOT_FLAG_NORMAL     0x0000

void write_boot_flag_soft_reset(void)
{
    RTC->BKP1R = BOOT_FLAG_SOFT_RESET; // BPK1R = DR1
}

void clear_boot_flag(void)
{
    RTC->BKP1R = 0;
}
```

```
uint8_t is_soft_reset(void)
{
    uint32_t val = RTC->BKP1R;
    return (val == BOOT_FLAG_SOFT_RESET);
}
```

[RTC 메모리를 이용한 flag 사용]

자율주행 차량 CAN Router 기반 Control Unit 개발

세부 개발 사항 5 - CRC 계산

- FW 소스 파일을 bin 파일로 빌드할때 CRC를 계산해 FW TAG 정보에 CRC 계산 값 추가 (좌측 이미지)
- BOOT mode에서 flash에 기록된 bin 파일의 CRC를 계산하고 해당 FW TAG의 CRC와 비교 검증함 (우측 이미지)

```
def stm32_crc32_hw_style(data: bytes) -> int:
    poly = 0x04C11DB7
    crc = 0xFFFFFFFF

    i = 0
    while i + 4 <= len(data):
        # 리틀엔디안으로 읽기
        word = data[i] | (data[i + 1] << 8) | (data[i + 2] << 16) | (data[i + 3] << 24)
        crc ^= word
        for _ in range(32):
            crc = ((crc << 1) ^ poly) & 0xFFFFFFFF if (crc & 0x80000000) else (crc << 1) & 0xFFFFFFFF
            i += 4

    remaining = len(data) - i
    if remaining:
        tail = data[i:] + b'\x00' * (4 - remaining)
        word = tail[0] | (tail[1] << 8) | (tail[2] << 16) | (tail[3] << 24)
        crc ^= word
        for _ in range(32):
            crc = ((crc << 1) ^ poly) & 0xFFFFFFFF if (crc & 0x80000000) else (crc << 1) & 0xFFFFFFFF

    return crc
```

```
uint32_t CAL_CRC(CRC_HandleTypeDef hcrc)
{
    HAL_CRC_DeInit(&hcrc);
    HAL_CRC_Init(&hcrc);
    __HAL_CRC_DR_RESET(&hcrc);
    uint32_t* bin_data = (uint32_t*)(USER_FLASH_FIRST_PAGE_ADDRESS + 0x400);
    uint32_t word_count = TAG_ADDRESS->size / 4; //TAG_ADDRESS->size
    uint32_t i;
    for (i = 0; i < word_count; i++)
    {
        hcrc.Instance->DR = bin_data[i];
    }

    uint32_t remaining = TAG_ADDRESS->size % 4; //TAG_ADDRESS->size
    if (remaining)
    {
        uint8_t* byte_ptr = (uint8_t*)&bin_data[word_count];
        for (i = 0; i < remaining; i++)
        {
            *((__IO uint8_t*)&hcrc.Instance->DR) = byte_ptr[i];
        }
    }
    return hcrc.Instance->DR;
}
```

세부 개발 사항 6 - Backup 시스템

- flash 영역 중 Backup 영역에 해당하는 내용을 현재 FW 영역으로 복사
→ CRC 비교 후 다르다면 Backup 영역의 이전 FW로 복구하는 코드 (하단 이미지)

```
252
253     if (TAG_ADDRESS->crc == crc_result){
254         udp_client_send("[BOOT] CRC same, Jump to new Fw\n");
255     }
256     else if (TAG_ADDRESS->crc != crc_result) {
257         udp_client_send("[BOOT] CRC not same, Back to before Fw\n");
258         HAL_FLASH_Unlock();
259         FLASH_Erase(USER_FLASH_FIRST_PAGE_ADDRESS);
260         Flash_Write_Address2 = USER_FLASH_FIRST_PAGE_ADDRESS;
261         uint32_t *src_data = (uint32_t*)BACKUP_FLASH_FIRST_PAGE_ADDRESS;
262         uint16_t data_len = ( BACKUP_TAG_ADDRESS->size + 1024 ) / 4;
263         uint32_t result = FLASH_Erase(Flash_Write_Address2, src_data ,data_len);
264     }
265 }
```

[Backup 진행 코드]

자율주행 차량 CAN Router 기반 Control Unit 개발

세부 개발 사항 7 - WDG 검출

- Bootmode 시작과 동시에 WDG 실행
- Bootmode에서 WDG으로 인한 Reset인지 감지 (하단 이미지)

```
if (__HAL_RCC_GET_FLAG(RCC_FLAG_IWDGRST))
{
    HAL_IWDG_Refresh(&hiwdg);
    udp_client_send("----CATCH WDG---- back to before FW\n");
    osDelay(500);
    WDGbackup();
}
```

세부 개발 사항 8 - CAN FD 통신

1. CAN Router를 통해 차량 내부 ECU 간 CAN BUS 내의 CAN 메시지 수집
2. Linux PC의 log 프로그램을 통해 CAN 메시지 수신
3. dbc 파일을 이용해 CAN ID 분석

→ 첨부된 동영상 파일 참고 : 운전석 문이 열리고 닫힘이 확인됨

https://youtu.be/_r_axqpXrdI

** CAN to Ethernet 변환 과정에서 지연이 발생하여 실시간성 저하 상황 추가 확인